

Лабораторная работа № 9

Автоматизация проверки Windows Server с помощью PowerShell

Продолжительность

90 минут.

Среда выполнения

- Windows Server;
 - PowerShell 5.1 или PowerShell 7;
 - Active Directory и DNS;
 - WinRM;
 - административная рабочая станция ADMIN01;
 - серверы FS01 и DC01;
 - Visual Studio Code или Windows PowerShell ISE.
-

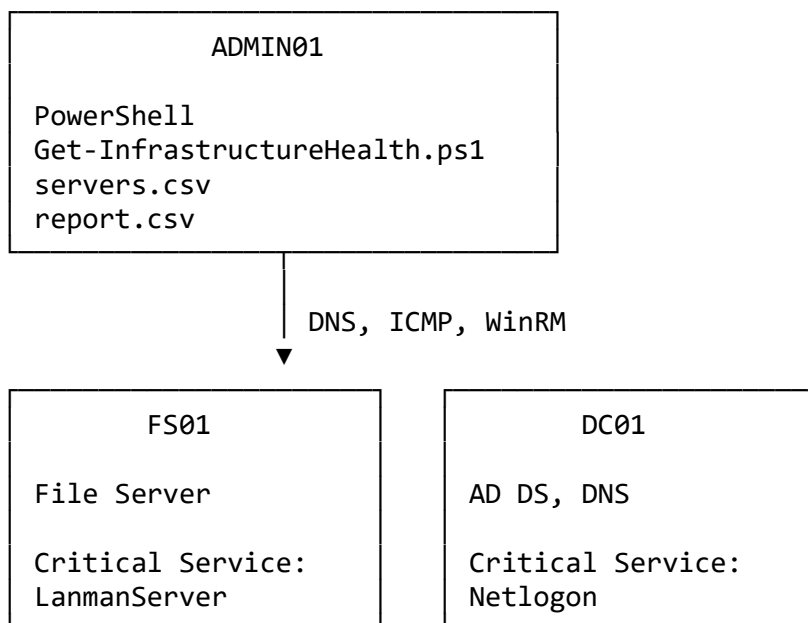
1. Цель работы

Научиться создавать простой PowerShell-скрипт для автоматизированной проверки нескольких Windows Server.

В ходе основной части необходимо:

- проверить объектный конвейер PowerShell;
 - подготовить список серверов в CSV;
 - проверить структуру входного файла;
 - проверить DNS;
 - проверить ICMP;
 - проверить WinRM;
 - получить сведения об операционной системе;
 - проверить свободное место на диске;
 - проверить критичную службу;
 - обработать ошибку недоступного сервера;
 - вернуть результаты в виде объектов;
 - присвоить статусы OK, WARNING, CRITICAL и UNKNOWN;
 - экспортировать отчёт в CSV.
-

2. Схема стенда



3. Структура проекта

```
C:\Scripts\InfrastructureHealth
├── Get-InfrastructureHealth.ps1
├── servers.csv
├── reports
│   └── infrastructure-health.csv
```

4. Формат входного файла

Файл `servers.csv` содержит:

```
Name,Role,CriticalService,DiskWarningPercent,DiskCriticalPercent
FS01,FileServer,LanmanServer,20,10
DC01,DomainController,Netlogon,20,10
```

Значение полей

Поле	Назначение
Name	DNS-имя сервера
Role	Роль сервера
CriticalService	Контролируемая служба

Поле	Назначение
DiskWarningPercent	Порог предупреждения
DiskCriticalPercent	Критичный порог

5. Модель статусов

Статус	Значение
OK	Проверка выполнена, отклонение не обнаружено
WARNING	Обнаружен риск или некритичное отклонение
CRITICAL	Подтверждён отказ или критичное состояние
UNKNOWN	Проверку выполнить не удалось или данных недостаточно

Статус UNKNOWN нельзя заменять на OK, если сервер не ответил или команда завершилась ошибкой.

6. Правила безопасности

- Скрипт основной части только читает состояние серверов.
 - В коде не хранятся пароли.
 - В отчёт не записываются Credentials.
 - Не используется TrustedHosts в доменной среде без необходимости.
 - Не применяется Format-Table перед Export-Csv.
 - Ошибка одного сервера не должна останавливать весь отчёт.
 - Все удалённые команды используют ограниченный набор read-only проверок.
 - Скрипт не перезапускает службы и не изменяет конфигурацию.
-

7. Базовые задания

Задание 1. Подготовить каталог проекта

На ADMIN01 открываем PowerShell от имени администратора.

Создаём структуру:

```
$ProjectPath = 'C:\Scripts\InfrastructureHealth'  
$ReportPath = Join-Path $ProjectPath 'reports'
```

```
New-Item `  
-ItemType Directory `  
-Path $ProjectPath `
```

```
-Force |  
Out-Null
```

```
New-Item `  
-ItemType Directory `  
-Path $ReportPath `  
-Force |  
Out-Null
```

Проверяем:

```
Get-ChildItem C:\Scripts\InfrastructureHealth
```

Задание 2. Проверить объектный конвейер

Получаем службы:

```
Get-Service |  
Where-Object Status -ne 'Running' |  
Select-Object Name,  
    DisplayName,  
    Status,  
    StartType
```

Проверяем тип объекта:

```
Get-Service |  
Select-Object -First 1 |  
Get-Member
```

Важно

Правильная последовательность:

получить объекты
→ отфильтровать
→ выбрать свойства
→ экспортировать

Неправильный вариант:

```
Get-Service |  
Format-Table |  
Export-Csv services.csv
```

Format-Table создаёт объекты форматирования, а не данные служб.

Задание 3. Создать servers.csv

Создаём файл:

```
@'
Name,Role,CriticalService,DiskWarningPercent,DiskCriticalPercent
FS01,FileServer,LanmanServer,20,10
DC01,DomainController,Netlogon,20,10
'@ |
    Set-Content `
        -Path "$ProjectPath\servers.csv" `
        -Encoding UTF8
```

Проверяем содержимое:

```
Get-Content "$ProjectPath\servers.csv"
```

Импортируем:

```
$Servers = Import-Csv "$ProjectPath\servers.csv"
```

Проверяем объекты:

```
$Servers |
    Format-Table
```

Проверяем свойства первой строки:

```
$Servers[0] |
    Get-Member
```

Задание 4. Проверить входной CSV

Проверяем наличие файла:

```
Test-Path "$ProjectPath\servers.csv"
```

Ожидается:

```
True
```

Проверяем количество строк:

```
$Servers.Count
```

Ожидается:

```
2
```

Проверяем обязательные заголовки:

```
$RequiredHeaders = @(
    'Name'
    'Role'
    'CriticalService'
    'DiskWarningPercent'
    'DiskCriticalPercent'
)
```

```
$ActualHeaders = $Servers[0].PSObject.Properties.Name
```

```
$MissingHeaders = $RequiredHeaders |  
    Where-Object {  
        $_ -notin $ActualHeaders  
    }
```

```
$MissingHeaders
```

При правильном CSV вывод должен быть пустым.

Задание 5. Проверить DNS серверов

Выполняем:

```
foreach ($Server in $Servers) {  
    Resolve-DnsName `  
        $Server.Name `  
        -ErrorAction SilentlyContinue  
}
```

Проверяем один сервер подробно:

```
Resolve-DnsName FS01
```

Ожидается IP-адрес сервера.

Если имя не разрешается, последующие проверки могут завершиться ошибкой.

Задание 6. Проверить ICMP

Выполняем:

```
foreach ($Server in $Servers) {  
    $PingResult = Test-Connection `  
        -ComputerName $Server.Name `  
        -Count 1 `  
        -Quiet  
  
    [PSCustomObject]@{  
        Server = $Server.Name  
        Ping   = $PingResult  
    }  
}
```

Пример результата:

Server Ping

```
-----  
FS01   True  
DC01   True
```

Ping показывает доступность ICMP, но не доказывает работу WinRM или серверной роли.

Задание 7. Проверить WinRM

Проверяем порт:

```
Test-NetConnection FS01 -Port 5985
```

Ожидается:

```
TcpTestSucceeded : True
```

Проверяем WS-Man:

```
Test-WSMan FS01
```

Повторяем для DC01:

```
Test-WSMan DC01
```

Если WinRM не настроен, на целевом сервере преподаватель выполняет:

```
Enable-PSRemoting -Force
```

В доменной среде обычно используется Kerberos, поэтому TrustedHosts не требуется.

Задание 8. Проверить удалённое выполнение

Выполняем:

```
Invoke-Command `
  -ComputerName FS01,DC01 `
  -ScriptBlock {
    [PSCustomObject]@{
      ComputerName = $env:COMPUTERNAME
      Date         = Get-Date
      PowerShell   = $PSVersionTable.PSVersion.ToString()
    }
  }
```

Результат должен содержать объекты от обоих серверов.

Задание 9. Создать функцию формирования результата

Открываем файл:

```
notepad "$ProjectPath\Get-InfrastructureHealth.ps1"
```

Добавляем начало скрипта:

```
[CmdletBinding()]  
param(  
    [Parameter()]  
    [string]$ServerList = (  
        Join-Path $PSScriptRoot 'servers.csv'  
    ),  
  
    [Parameter()]  
    [string]$OutputPath = (  
        Join-Path $PSScriptRoot 'reports'  
    )  
)
```

```
Set-StrictMode -Version Latest  
$ErrorActionPreference = 'Stop'
```

```
function New-HealthResult {  
    [CmdletBinding()]  
    param(  
        [Parameter(Mandatory)]  
        [string]$Server,  
  
        [Parameter(Mandatory)]  
        [string]$Role,  
  
        [Parameter(Mandatory)]  
        [string]$Check,  
  
        [Parameter(Mandatory)]  
        [ValidateSet(  
            'OK',  
            'WARNING',  
            'CRITICAL',  
            'UNKNOWN'  
        )]  
        [string]$Status,  
  
        [Parameter()]  
        [string]$Value,  
  
        [Parameter()]  
        [string]$Threshold,
```

```

        [Parameter()]
        [string]$Details
    )

    [PSCustomObject]@(
        Server      = $Server
        Role        = $Role
        Check       = $Check
        Status      = $Status
        Value       = $Value
        Threshold   = $Threshold
        Details     = $Details
        Timestamp   = Get-Date
    )
}

```

Функция возвращает объект, а не выводит текст через Write-Host.

Задание 10. Добавить проверку CSV

Продолжаем скрипт:

```

if (-not (Test-Path $ServerList)) {
    throw "Server list not found: $ServerList"
}

$Servers = Import-Csv -Path $ServerList

if (-not $Servers) {
    throw 'Server list is empty.'
}

$RequiredHeaders = @(
    'Name'
    'Role'
    'CriticalService'
    'DiskWarningPercent'
    'DiskCriticalPercent'
)

$ActualHeaders = $Servers[0].PSObject.Properties.Name

foreach ($Header in $RequiredHeaders) {
    if ($Header -notin $ActualHeaders) {
        throw "Missing required CSV header: $Header"
    }
}

if (-not (Test-Path $OutputPath)) {
    New-Item `

```

```

        -ItemType Directory `
        -Path $OutputPath `
        -Force |
        Out-Null
    }

$Results = [System.Collections.Generic.List[object]]::new()

```

Задание 11. Добавить проверку DNS

В основной цикл добавляем:

```

foreach ($Server in $Servers) {
    $ServerName = $Server.Name
    $Role = $Server.Role

    try {
        $DnsResult = Resolve-DnsName `
            -Name $ServerName `
            -ErrorAction Stop |
            Where-Object IPAddress |
            Select-Object -First 1

        $Results.Add(
            (
                New-HealthResult `
                    -Server $ServerName `
                    -Role $Role `
                    -Check 'DNS' `
                    -Status 'OK' `
                    -Value $DnsResult.IPAddress `
                    -Details 'DNS name resolved.'
            )
        )
    }
    catch {
        $Results.Add(
            (
                New-HealthResult `
                    -Server $ServerName `
                    -Role $Role `
                    -Check 'DNS' `
                    -Status 'CRITICAL' `
                    -Details $_.Exception.Message
            )
        )
    }
}

```

На этом этапе скрипт уже создаёт структурированный объект для каждого сервера.

Задание 12. Добавить проверку ICMP

Внутри того же цикла, после DNS:

```
try {
    $PingOk = Test-Connection `
        -ComputerName $ServerName `
        -Count 1 `
        -Quiet `
        -ErrorAction Stop

    if ($PingOk) {
        $PingStatus = 'OK'
        $PingDetails = 'ICMP reply received.'
    }
    else {
        $PingStatus = 'WARNING'
        $PingDetails = 'No ICMP reply.'
    }

    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'ICMP' `
                -Status $PingStatus `
                -Value $PingOk `
                -Details $PingDetails
        )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'ICMP' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
        )
    )
}
```

Отсутствие ответа Ping не всегда означает отказ сервера, поэтому используется WARNING, а не автоматический CRITICAL.

Задание 13. Добавить проверку WinRM

```
$WinRmAvailable = $false

try {
    Test-WSMan `
        -ComputerName $ServerName `
        -ErrorAction Stop |
        Out-Null

    $WinRmAvailable = $true

    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'WinRM' `
                -Status 'OK' `
                -Value 'TCP 5985' `
                -Details 'WS-Man responded.'
        )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'WinRM' `
                -Status 'CRITICAL' `
                -Value 'Unavailable' `
                -Details $_.Exception.Message
        )
    )
}
```

Переменная \$WinRmAvailable используется для зависимых проверок.

Задание 14. Добавить проверку операционной системы

Если WinRM доступен:

```
if ($WinRmAvailable) {
    try {
        $OperatingSystem = Get-CimInstance `
            -ClassName Win32_OperatingSystem `
            -ComputerName $ServerName `
            -ErrorAction Stop
    }
```

```

        $Results.Add(
            (
                New-HealthResult `
                    -Server $ServerName `
                    -Role $Role `
                    -Check 'OperatingSystem' `
                    -Status 'OK' `
                    -Value $OperatingSystem.Caption `
                    -Details (
                        "Version: {0}; LastBoot: {1}" -f
                        $OperatingSystem.Version,
                        $OperatingSystem.LastBootUpTime
                    )
            )
        )
    }
    catch {
        $Results.Add(
            (
                New-HealthResult `
                    -Server $ServerName `
                    -Role $Role `
                    -Check 'OperatingSystem' `
                    -Status 'UNKNOWN' `
                    -Details $_.Exception.Message
            )
        )
    }
}
else {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'OperatingSystem' `
                -Status 'UNKNOWN' `
                -Details 'Skipped because WinRM is unavailable.'
        )
    )
}
}

```

Задание 15. Добавить проверку диска C:

Внутри условия доступного WinRM:

```

if ($WinRmAvailable) {
    try {
        $Disk = Get-CimInstance `

```

```

        -ClassName Win32_LogicalDisk `
        -ComputerName $ServerName `
        -Filter "DeviceID='C:'" `
        -ErrorAction Stop

    if (-not $Disk) {
        throw 'Disk C: was not returned.'
    }

    $FreePercent = [math]::Round(
        ($Disk.FreeSpace / $Disk.Size) * 100,
        2
    )

    $FreeGB = [math]::Round(
        $Disk.FreeSpace / 1GB,
        2
    )

    $WarningPercent = [int]$Server.DiskWarningPercent
    $CriticalPercent = [int]$Server.DiskCriticalPercent

    if (
        $FreePercent -lt $CriticalPercent -or
        $FreeGB -lt 10
    ) {
        $DiskStatus = 'CRITICAL'
    }
    elseif (
        $FreePercent -lt $WarningPercent -or
        $FreeGB -lt 20
    ) {
        $DiskStatus = 'WARNING'
    }
    else {
        $DiskStatus = 'OK'
    }

    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'Disk C:' `
                -Status $DiskStatus `
                -Value "$FreePercent% / $FreeGB GB free" `
                -Threshold (
                    "Warning < $WarningPercent%; " +
                    "Critical < $CriticalPercent%"
                ) `
                -Details 'System disk free space.'
        )
    )

```

```

    )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'Disk C:' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
        )
    )
}
}
else {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'Disk C:' `
                -Status 'UNKNOWN' `
                -Details 'Skipped because WinRM is unavailable.'
        )
    )
}
}

```

Диск оценивается одновременно по проценту и абсолютному остатку в гигабайтах.

Задание 16. Добавить проверку критичной службы

```

if ($WinRmAvailable) {
    try {
        $ServiceName = $Server.CriticalService

        $Service = Get-CimInstance `
            -ClassName Win32_Service `
            -ComputerName $ServerName `
            -Filter "Name='$ServiceName'" `
            -ErrorAction Stop

        if (-not $Service) {
            throw "Service not found: $ServiceName"
        }

        if ($Service.State -eq 'Running') {
            $ServiceStatus = 'OK'
        }
    }
}

```

```

else {
    $ServiceStatus = 'CRITICAL'
}

$Results.Add(
    (
        New-HealthResult `
            -Server $ServerName `
            -Role $Role `
            -Check "Service $ServiceName" `
            -Status $ServiceStatus `
            -Value $Service.State `
            -Threshold 'Desired state: Running' `
            -Details (
                "StartMode: {0}" -f
                $Service.StartMode
            )
        )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'CriticalService' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
            )
        )
    )
}
else {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'CriticalService' `
                -Status 'UNKNOWN' `
                -Details 'Skipped because WinRM is unavailable.'
            )
        )
    )
}
}

```

Задание 17. Экспортировать CSV-отчёт

После завершения цикла добавляем:

```

$Timestamp = Get-Date -Format 'yyyyMMdd-HH:mm:ss'

$CsvPath = Join-Path `
    $OutputPath `
    "infrastructure-health-$Timestamp.csv"

$Results |
    Export-Csv `
        -Path $CsvPath `
        -NoTypeInfoInformation `
        -Encoding UTF8

$Results |
    Sort-Object Server, Check |
    Format-Table `
        Server,
        Role,
        Check,
        Status,
        Value `
        -AutoSize

Write-Host "Report created: $CsvPath"

```

Format-Table применяется только после формирования и экспорта объектов.

8. Полный скрипт

```

[CmdletBinding()]
param(
    [Parameter()]
    [string]$ServerList = (
        Join-Path $PSScriptRoot 'servers.csv'
    ),

    [Parameter()]
    [string]$OutputPath = (
        Join-Path $PSScriptRoot 'reports'
    )
)

Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function New-HealthResult {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Server,

```

```

        [Parameter(Mandatory)]
        [string]$Role,

        [Parameter(Mandatory)]
        [string]$Check,

        [Parameter(Mandatory)]
        [ValidateSet(
            'OK',
            'WARNING',
            'CRITICAL',
            'UNKNOWN'
        )]
        [string]$Status,

        [Parameter()]
        [string]$Value,

        [Parameter()]
        [string]$Threshold,

        [Parameter()]
        [string]$Details
    )

    [PSCustomObject]@{
        Server      = $Server
        Role        = $Role
        Check       = $Check
        Status      = $Status
        Value       = $Value
        Threshold   = $Threshold
        Details     = $Details
        Timestamp   = Get-Date
    }
}

if (-not (Test-Path $ServerList)) {
    throw "Server list not found: $ServerList"
}

$Servers = Import-Csv -Path $ServerList

if (-not $Servers) {
    throw 'Server list is empty.'
}

$RequiredHeaders = @(
    'Name'
    'Role'

```

```

        'CriticalService'
        'DiskWarningPercent'
        'DiskCriticalPercent'
    )

    $ActualHeaders = $Servers[0].PSObject.Properties.Name

    foreach ($Header in $RequiredHeaders) {
        if ($Header -notin $ActualHeaders) {
            throw "Missing required CSV header: $Header"
        }
    }

    if (-not (Test-Path $OutputPath)) {
        New-Item `
            -ItemType Directory `
            -Path $OutputPath `
            -Force |
        Out-Null
    }

    $Results = [System.Collections.Generic.List[object]]::new()

    foreach ($Server in $Servers) {
        $ServerName = $Server.Name
        $Role = $Server.Role
        $WinRmAvailable = $false

        try {
            $DnsResult = Resolve-DnsName `
                -Name $ServerName `
                -ErrorAction Stop |
            Where-Object IPAddress |
            Select-Object -First 1

            $Results.Add(
                (
                    New-HealthResult `
                        -Server $ServerName `
                        -Role $Role `
                        -Check 'DNS' `
                        -Status 'OK' `
                        -Value $DnsResult.IPAddress `
                        -Details 'DNS name resolved.'
                )
            )
        }
        catch {
            $Results.Add(
                (
                    New-HealthResult `

```

```

        -Server $ServerName `
        -Role $Role `
        -Check 'DNS' `
        -Status 'CRITICAL' `
        -Details $_.Exception.Message
    )
}

try {
    $PingOk = Test-Connection `
        -ComputerName $ServerName `
        -Count 1 `
        -Quiet `
        -ErrorAction Stop

    if ($PingOk) {
        $PingStatus = 'OK'
        $PingDetails = 'ICMP reply received.'
    }
    else {
        $PingStatus = 'WARNING'
        $PingDetails = 'No ICMP reply.'
    }

    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'ICMP' `
                -Status $PingStatus `
                -Value $PingOk `
                -Details $PingDetails
        )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'ICMP' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
        )
    )
}

try {
    Test-WSMan `

```

```

-ComputerName $ServerName `
-ErrorAction Stop |
Out-Null

$WinRmAvailable = $true

$Results.Add(
(
    New-HealthResult `
        -Server $ServerName `
        -Role $Role `
        -Check 'WinRM' `
        -Status 'OK' `
        -Value 'TCP 5985' `
        -Details 'WS-Man responded.'
    )
)
}
catch {
    $Results.Add(
    (
        New-HealthResult `
            -Server $ServerName `
            -Role $Role `
            -Check 'WinRM' `
            -Status 'CRITICAL' `
            -Value 'Unavailable' `
            -Details $_.Exception.Message
        )
    )
}

if ($WinRmAvailable) {
    try {
        $OperatingSystem = Get-CimInstance `
            -ClassName Win32_OperatingSystem `
            -ComputerName $ServerName `
            -ErrorAction Stop

        $Results.Add(
            (
                New-HealthResult `
                    -Server $ServerName `
                    -Role $Role `
                    -Check 'OperatingSystem' `
                    -Status 'OK' `
                    -Value $OperatingSystem.Caption `
                    -Details (
                        "Version: {0}; LastBoot: {1}" -f
                        $OperatingSystem.Version,
                        $OperatingSystem.LastBootUpTime
                    )
            )
        )
    }
}

```

```

    )
  )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'OperatingSystem' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
        )
    )
}

try {
    $Disk = Get-CimInstance `
        -ClassName Win32_LogicalDisk `
        -ComputerName $ServerName `
        -Filter "DeviceID='C:'" `
        -ErrorAction Stop

    if (-not $Disk) {
        throw 'Disk C: was not returned.'
    }

    $FreePercent = [math]::Round(
        ($Disk.FreeSpace / $Disk.Size) * 100,
        2
    )

    $FreeGB = [math]::Round(
        $Disk.FreeSpace / 1GB,
        2
    )

    $WarningPercent = [int]$Server.DiskWarningPercent
    $CriticalPercent = [int]$Server.DiskCriticalPercent

    if (
        $FreePercent -lt $CriticalPercent -or
        $FreeGB -lt 10
    ) {
        $DiskStatus = 'CRITICAL'
    }
    elseif (
        $FreePercent -lt $WarningPercent -or
        $FreeGB -lt 20
    ) {
        $DiskStatus = 'WARNING'
    }
}

```

```

    }
    else {
        $DiskStatus = 'OK'
    }

    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'Disk C:' `
                -Status $DiskStatus `
                -Value "$FreePercent% / $FreeGB GB free" `
                -Threshold (
                    "Warning < $WarningPercent%; " +
                    "Critical < $CriticalPercent%"
                ) `
                -Details 'System disk free space.'
        )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'Disk C:' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
        )
    )
}

try {
    $ServiceName = $Server.CriticalService

    $Service = Get-CimInstance `
        -ClassName Win32_Service `
        -ComputerName $ServerName `
        -Filter "Name='$ServiceName'" `
        -ErrorAction Stop

    if (-not $Service) {
        throw "Service not found: $ServiceName"
    }

    if ($Service.State -eq 'Running') {
        $ServiceStatus = 'OK'
    }
    else {
        $ServiceStatus = 'CRITICAL'
    }
}

```

```
}

$Results.Add(
    (
        New-HealthResult `
            -Server $ServerName `
            -Role $Role `
            -Check "Service $ServiceName" `
            -Status $ServiceStatus `
            -Value $Service.State `
            -Threshold 'Desired state: Running' `
            -Details (
                "StartMode: {0}" -f
                $Service.StartMode
            )
        )
    )
}
catch {
    $Results.Add(
        (
            New-HealthResult `
                -Server $ServerName `
                -Role $Role `
                -Check 'CriticalService' `
                -Status 'UNKNOWN' `
                -Details $_.Exception.Message
            )
        )
    }
}
else {
    foreach ($CheckName in @(
        'OperatingSystem',
        'Disk C:',
        'CriticalService'
    )) {
        $Results.Add(
            (
                New-HealthResult `
                    -Server $ServerName `
                    -Role $Role `
                    -Check $CheckName `
                    -Status 'UNKNOWN' `
                    -Details (
                        'Skipped because WinRM is unavailable.'
                    )
                )
            )
        }
    }
}
```

```

$Timestamp = Get-Date -Format 'yyyyMMdd-HH:mm:ss'

$CsvPath = Join-Path `
    $OutputPath `
    "infrastructure-health-$Timestamp.csv"

$Results |
    Export-Csv `
        -Path $CsvPath `
        -NoTypeInfoInformation `
        -Encoding UTF8

$Results |
    Sort-Object Server, Check |
    Format-Table `
        Server,
        Role,
        Check,
        Status,
        Value `
        -AutoSize

Write-Host "Report created: $CsvPath"

```

9. Проверка синтаксиса

Сохраняем скрипт.

Проверяем синтаксис:

```

$ParseErrors = $null

[System.Management.Automation.Language.Parser]::ParseFile(
    "$ProjectPath\Get-InfrastructureHealth.ps1",
    [ref]$null,
    [ref]$ParseErrors
) | Out-Null

$ParseErrors

```

При правильном синтаксисе вывод пустой.

10. Запуск скрипта

Переходим в каталог проекта:

`Set-Location C:\Scripts\InfrastructureHealth`

Запускаем:

```
.\Get-InfrastructureHealth.ps1
```

Или с явными параметрами:

```
.\Get-InfrastructureHealth.ps1 `
  -ServerList C:\Scripts\InfrastructureHealth\servers.csv `
  -OutputPath C:\Scripts\InfrastructureHealth\reports
```

Ожидается таблица:

Server	Role	Check	Status	Value
-----	----	-----	-----	-----
DC01	DomainController	DNS	OK	192.168...
DC01	DomainController	Disk C:	OK	...
DC01	DomainController	ICMP	OK	True
DC01	DomainController	OperatingSystem	OK	Windows...
DC01	DomainController	Service Netlogon	OK	Running
DC01	DomainController	WinRM	OK	TCP 5985
FS01	FileServer	DNS	OK	192.168...
FS01	FileServer	Disk C:	OK	...
FS01	FileServer	ICMP	OK	True
FS01	FileServer	OperatingSystem	OK	Windows...
FS01	FileServer	Service LanmanServer	OK	Running
FS01	FileServer	WinRM	OK	TCP 5985

11. Проверить CSV-отчёт

Выводим созданные файлы:

```
Get-ChildItem `
  C:\Scripts\InfrastructureHealth\reports |
  Sort-Object LastWriteTime -Descending
```

Открываем последний CSV:

```
$LatestReport = Get-ChildItem `
  C:\Scripts\InfrastructureHealth\reports `
  -Filter *.csv |
  Sort-Object LastWriteTime -Descending |
  Select-Object -First 1
```

```
Import-Csv $LatestReport.FullName |
  Format-Table
```

Проверяем обязательные поля:

Server
Role
Check
Status
Value
Threshold
Details
Timestamp

12. Ожидаемый результат базовой части

К концу лабораторной работы:

- создан `servers.csv`;
 - входной CSV проверяется;
 - скрипт использует параметры;
 - результаты формируются как `PSCustomObject`;
 - DNS проверяется отдельно;
 - ICMP проверяется отдельно;
 - WinRM проверяется отдельно;
 - сведения об ОС получаются через CIM;
 - диск оценивается по проценту и гигабайтам;
 - критичная служба выбирается из CSV;
 - зависимые проверки получают UNKNOWN, если WinRM недоступен;
 - ошибка одного сервера не останавливает весь отчёт;
 - создан структурированный CSV.
-

13. Дополнительные задания

Дополнительное задание 1. Создать HTML- и JSON-отчёты

После формирования `$Results` добавить:

```
$HtmlPath = Join-Path `
    $OutputPath `
    "infrastructure-health-$Timestamp.html"
```

```
$JsonPath = Join-Path `
    $OutputPath `
    "infrastructure-health-$Timestamp.json"
```

HTML:

```
$Results |
    Sort-Object Server, Check |
    ConvertTo-Html `
        -Title 'Infrastructure Health' `
        -PreContent '<h1>Infrastructure Health</h1>' |
    Set-Content `
        -Path $HtmlPath `
        -Encoding UTF8
```

JSON:

```
$Results |
    ConvertTo-Json -Depth 5 |
    Set-Content `
        -Path $JsonPath `
        -Encoding UTF8
```

Проверить:

```
Start-Process $HtmlPath
Get-Content $JsonPath -First 20
```

Все форматы должны строиться из одного массива \$Results.

Дополнительное задание 2. Добавить operational log

Создать функцию:

```
function Write-Log {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory)]
        [string]$Message,

        [Parameter(Mandatory)]
        [ValidateSet('INFO', 'WARNING', 'ERROR')]
        [string]$Level,

        [Parameter(Mandatory)]
        [string]$Path
    )

    $Timestamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ssK'

    "$Timestamp [$Level] $Message" |
        Add-Content `
            -Path $Path `
            -Encoding UTF8
}
```

Пример:

```
$LogPath = Join-Path `
    $OutputPath `
    "infrastructure-health-$Timestamp.log"
```

```
Write-Log `
    -Message 'Script started.' `
    -Level INFO `
    -Path $LogPath
```

Operational Log описывает ход работы скрипта, а Health Report — состояние серверов.

Дополнительное задание 3. Смоделировать недоступный WinRM

На тестовом сервере FS01 временно остановить WinRM:

```
Stop-Service WinRM
```

Запустить скрипт с ADMIN01.

Ожидается:

Проверка	Статус
DNS	По фактическому результату
ICMP	По фактическому результату
WinRM	CRITICAL
OperatingSystem	UNKNOWN
Disk C:	UNKNOWN
CriticalService	UNKNOWN

После теста запустить WinRM:

```
Start-Service WinRM
```

Повторить отчёт.

Все проверки должны вернуться к фактическим значениям.

Дополнительное задание 4. Смоделировать отказ службы

На FS01 остановить безопасную службу:

```
Stop-Service Spooler
```

Добавить в servers.csv строку:

```
FS01,FileServer,Spooler,20,10
```

Повторить отчёт.

Ожидается:

Service Spooler : CRITICAL

Остальные серверы и проверки должны продолжить работу.

После теста:

`Start-Service Spooler`

Вернуть исходную критичную службу LanmanServer.

Дополнительное задание 5. Добавить проверку событий

Для доступного сервера выполнять:

```
$Events = Invoke-Command `
    -ComputerName $ServerName `
    -ScriptBlock {
        Get-WinEvent `
            -FilterHashtable @{
                LogName     = 'System'
                Level       = 1,2
                StartTime   = (Get-Date).AddHours(-24)
            } `
            -ErrorAction SilentlyContinue
    }
```

Если события найдены:

WARNING или CRITICAL

в зависимости от согласованной политики.

Не следует делать весь сервер CRITICAL из-за старой ошибки вне анализируемого периода.

Дополнительное задание 6. Добавить DurationMs

Перед каждой проверкой:

```
$Stopwatch = [System.Diagnostics.Stopwatch]::StartNew()
```

После выполнения:

```
$Stopwatch.Stop()
$DurationMs = $Stopwatch.ElapsedMilliseconds
```

Добавить поле:

DurationMs

в функцию New-HealthResult.

Это позволяет найти медленные DNS-, WinRM- и CIM-запросы.

Дополнительное задание 7. Настроить Task Scheduler

Создать каталог:

C:\Scripts\InfrastructureHealth

Program:

powershell.exe

Arguments:

```
-NoProfile -NonInteractive -ExecutionPolicy RemoteSigned -File  
"C:\Scripts\InfrastructureHealth\Get-InfrastructureHealth.ps1" -ServerList  
"C:\Scripts\InfrastructureHealth\servers.csv" -OutputPath  
"C:\Scripts\InfrastructureHealth\reports"
```

Start in:

C:\Scripts\InfrastructureHealth

Проверить:

```
Get-ScheduledTaskInfo `
    -TaskName 'Infrastructure Health'
```

Фиксируются:

- LastRunTime;
- LastTaskResult;
- NextRunTime;
- NumberOfMissedRuns.

Пароль не должен находиться в коде или аргументах задачи.

Дополнительное задание 8. Подготовить Git-репозиторий

Структура:

```
Get-InfrastructureHealth.ps1  
servers.csv  
README.md  
.gitignore  
reports\  
logs\
```

Пример .gitignore:

```
reports/  
logs/  
*.clixml  
*.tmp  
*.log  
config.private.psd1
```

В Git сохраняются:

- код;
- безопасный пример CSV;
- README;
- .gitignore.

Не сохраняются:

- Credentials;
- отчёты;
- логи;
- Transcripts;
- приватная конфигурация.

14. Типовые ошибки

Симптом	Причина	Что проверить
CSV не импортируется	Неверный разделитель или кодировка	Заголовки и Import-Csv
catch не выполняется	Нет -ErrorAction Stop	Параметры командлета
Отчёт содержит данные форматирования	Format-Table до Export-Csv	Порядок pipeline
CIM не подключается	WinRM или Firewall	Test-WSMan, TCP 5985
Ping не проходит, WinRM работает	ICMP запрещён Firewall	Не считать сервер недоступным
Служба Stopped, но это нормально	Manual или Trigger Start	StartMode и роль
Весь скрипт остановился	Нет локального try/catch	Независимые проверки
Путь CSV не найден в Task Scheduler	Неверная рабочая папка	\$PSScriptRoot, Start in

15. Способы проверки

Проверка синтаксиса

```
[System.Management.Automation.Language.Parser]::ParseFile(  
    'C:\Scripts\InfrastructureHealth\Get-InfrastructureHealth.ps1',  
    [ref]$null,  
    [ref]$ParseErrors  
) | Out-Null
```

Проверка DNS

```
Resolve-DnsName FS01
```

Проверка WinRM

```
Test-WSMan FS01
```

Проверка удалённой команды

```
Invoke-Command `   
    -ComputerName FS01 `   
    -ScriptBlock {   
        hostname   
    }   
    }
```

Проверка отчёта

```
Import-Csv `   
    C:\Scripts\InfrastructureHealth\reports\<ИМЯ_ОТЧЁТА>.csv |   
    Group-Object Status
```

16. Приёмочная матрица

№	Проверка	Ожидаемый результат	Результат
1	Каталог проекта	Создан	
2	servers.csv	Создан	
3	CSV Headers	Проверены	
4	CSV Rows	Не пустые	
5	DNS FS01	Проверен	
6	DNS DC01	Проверен	
7	ICMP	Проверен отдельно	
8	WinRM FS01	Доступен	
9	WinRM DC01	Доступен	
10	CIM OS	Получены данные	
11	Disk C:	Получен статус	

№	Проверка	Ожидаемый результат	Результат
12	Critical Service	Получен статус	
13	Результаты	PSCustomObject	
14	Статусы	Используются четыре значения	
15	Ошибки	Обрабатываются через catch	
16	UNKNOWN	Используется для пропущенных проверок	
17	CSV Report	Создан	
18	Secrets	Отсутствуют	

17. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему стенда.
3. Структуру проекта.
4. Содержимое `servers.csv`.
5. Объяснение объектного pipeline.
6. Результат проверки заголовков CSV.
7. Результат DNS-проверки.
8. Результат ICMP.
9. Результат WinRM.
10. Сведения об ОС серверов.
11. Состояние диска C:.
12. Состояние критичных служб.
13. Фрагмент функции `New-HealthResult`.
14. Пример объекта результата.
15. Фрагмент обработки `try/catch`.
16. Статусы OK, WARNING, CRITICAL, UNKNOWN.
17. Созданный CSV.
18. Заполненную приёмочную матрицу.
19. Краткий вывод.

Для дополнительных заданий включаются только фактически выполненные результаты.

В отчёт запрещено включать:

- пароли;
- Credential XML;
- токены;
- приватные ключи;

- содержимое защищённых конфигураций;
 - полные Transcripts с секретными командами.
-

18. Чек-лист выполнения

Базовая часть

- ☐ Создан каталог проекта.
- ☐ Создан каталог reports.
- ☐ Создан servers.csv.
- ☐ Проверены заголовки CSV.
- ☐ Проверено количество серверов.
- ☐ Изучен объектный pipeline.
- ☐ Проверена команда Get-Member.
- ☐ Проверен DNS.
- ☐ Проверен ICMP.
- ☐ Проверен TCP 5985.
- ☐ Проверен Test-WSMan.
- ☐ Проверен Invoke-Command.
- ☐ Создан параметризованный скрипт.
- ☐ Создана функция New-HealthResult.
- ☐ Добавлена проверка DNS.
- ☐ Добавлена проверка ICMP.
- ☐ Добавлена проверка WinRM.
- ☐ Добавлена проверка ОС.
- ☐ Добавлена проверка диска.
- ☐ Добавлена проверка службы.
- ☐ Добавлена обработка ошибок.
- ☐ Используется статус UNKNOWN.
- ☐ Создан CSV-отчёт.
- ☐ В коде нет паролей.

Дополнительная часть

- ☐ Создан HTML-отчёт.
- ☐ Создан JSON-отчёт.
- ☐ Создан operational log.
- ☐ Проверен отказ WinRM.
- ☐ Проверен отказ службы.

- ☐ Добавлена проверка событий.
 - ☐ Добавлено поле DurationMs.
 - ☐ Настроен Task Scheduler.
 - ☐ Подготовлен Git-репозиторий.
-

19. Контрольные вопросы

1. Чем объектный pipeline PowerShell отличается от текстового конвейера?
2. Почему Format-Table нельзя использовать перед Export-Csv?
3. Чем Test-Connection отличается от Test-WSMan?
4. Для чего используется Get-CimInstance?
5. Почему для try/catch часто требуется -ErrorAction Stop?
6. Для чего нужен статус UNKNOWN?
7. Почему проверка свободного места должна учитывать проценты и гигабайты?
8. Почему Health Check не должен автоматически исправлять найденные проблемы?